

FIT2099 Assignment 3 Rationale (REQ4)

by Edwin Chuah

Note that there will be a combined document of REQ1-5 for the design rationales. It is advised for the marker to check there instead, though the REQ4 rationale should be the same for both documents.

The UML class diagram for REQ4 is as follows:

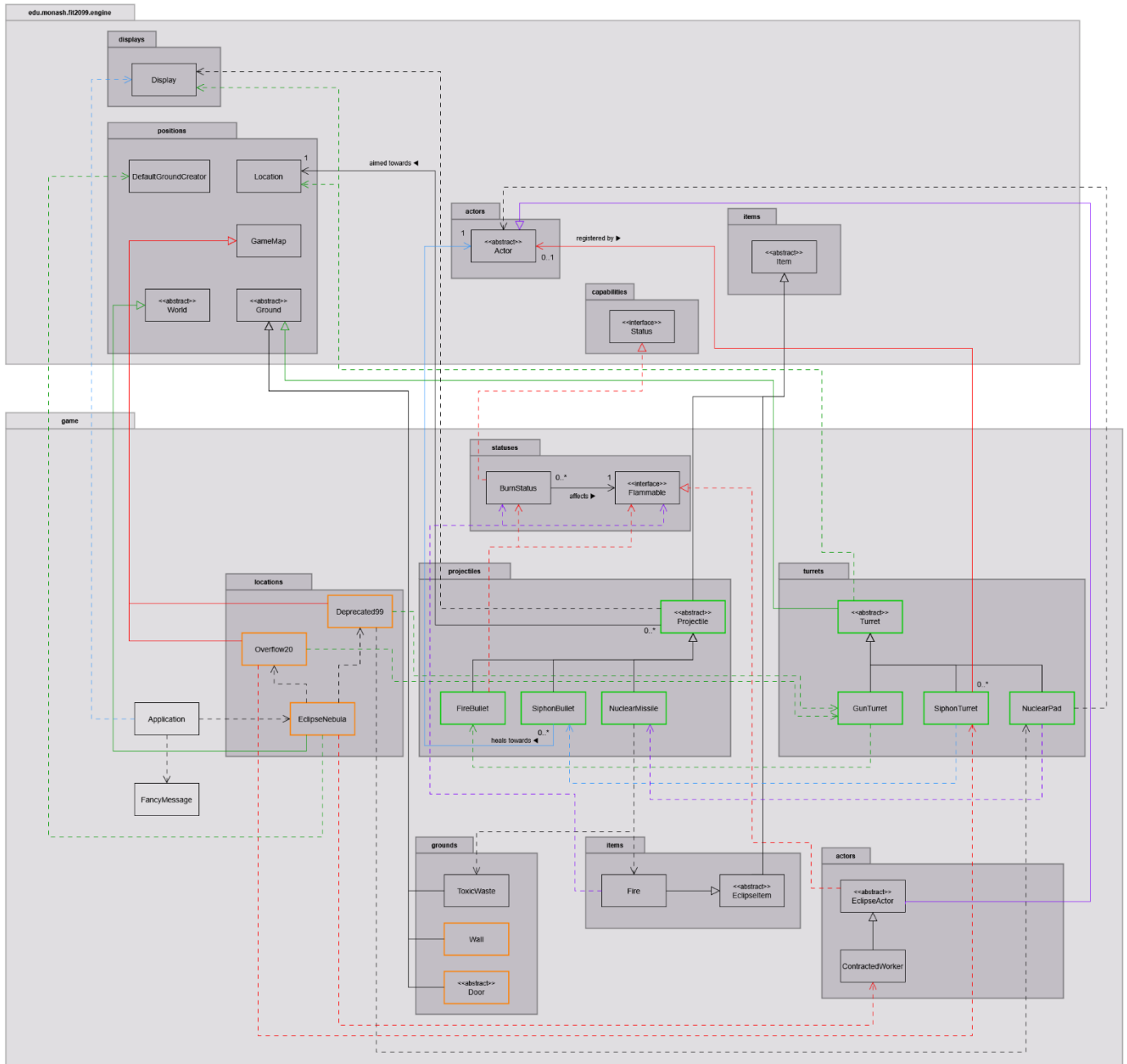
Notes:

Overlapping lines are given different colours for clarity.

Classes may have different colours on their outline:
Black classes are already existing classes (either from A2/A1 or a previous REQ) that have not been changed for this requirement.

Orange classes are existing classes that have been changed for this requirement.
Green classes are new classes for this requirement.

Walls and Doors have also been changed for this requirement to block Projectiles.



This design rationale aims to cover the major decisions I've made across Requirement 4 (Creative Mode).

Decision 1a: Creating the turrets as per the proposal.

Chosen design: Create an abstract class Turret, which inherits Ground, and have the proposed turrets inherit Turret.

Advantages	Disadvantages
Allows standardizing the common behaviour of turrets (detecting enemies, determining when a projectile should be fired etc.)	Adds a new abstraction layer, increasing the number of layers the turrets are tightly coupled to (Turret, Ground etc.)

Alternative design: No Turret abstract class. Instead, the proposed turrets will just extend Ground directly.

Advantages	Disadvantages
Avoids adding a new abstraction layer, minimizing the number of layers each turret is tightly coupled with.	Common behaviour will need to be repeated across each turret concrete class.

The chosen design was used as each proposed turret shares common behaviour (e.g. detecting enemies) as well as common statistics (e.g. ammunition capacity). By having those standardized in the Turret abstract class, unnecessary code duplication, which is a code smell, is avoided, adhering to DRY. This makes it more maintainable as any changes to common behaviour will only need to be changed once rather than for each existing type of turret, which justifies having the additional abstraction layer.

Similarly, Projectile was also created as an abstract class with the same reasoning of mainly adhering to DRY.

Also, the Creative Mode requirement explicitly requires 2 abstractions, so the alternative design would've not even met the requirement.

Decision 1b: Should Projectile directly extend Item or Eclipseltem?

The chosen design was to directly extend Item. Eclipseltem's purpose is to standardize the game's items to have a weight and portable ability. Since Projectile is non-portable (as it's fired and moving) and does not need a weight, there's no point in extending Eclipseltem just to remove the portable ability and make it 0 weight. By directly extending Item, this removes an unnecessary layer of abstraction as Projectile would not be tightly coupled with Eclipseltem, which does adhere to KISS. This also avoids the code smell of Refused Bequest (as it would remove Eclipseltem's portability), as well as another code smell of having too deep inheritance.

Decision 2a: Placing numerical values of statistics and effects in the classes of this requirement (e.g. GunTurret's ammo capacity and FireBullet's velocity).

The chosen design was to define those values as static final constants (e.g. AMMO_CAPACITY, VELOCITY and HIT_DAMAGE). The alternative design would be to simply not have those constants and have numeric literals instead (e.g. actor.hurt(2) instead of actor.hurt(HIT_DAMAGE)).

The chosen design was used as it eliminates magic numbers, weakening the connascence of meaning into connascence of name. Furthermore, this avoids the code smell of having excessive literals/magic numbers. This would improve readability especially for other, new people working on the code since

these values now have a meaning attached to them. This also improves maintainability in case the same value (e.g. HIT_DAMAGE) would need to be used multiple times in the class. If said value would need to be changed, it only needs to be changed once (just change the value of the constant) rather than needing to find and change every occurrence of it.

Decision 2b: Structuring the tick method of Turret.

Chosen design: Split different parts of the logic (detecting enemies, determining if ready to fire etc.) into their own method.

Advantages	Disadvantages
This would be more readable.	May need to be more careful about the order in which methods are called (e.g. getTargetDestination must be called before fireProjectileAt).
Allows subclasses to override some methods (e.g. extra conditions for isReady method) without having to rewrite logic for the other methods.	-

Alternative design: Keep all of the logic in the tick method.

Advantages	Disadvantages
Seems simpler to just keep all of the logic in one method.	Less readable and maintainable as making changes would require finding the specific part of the logic in the long method to change.
-	Requires rewriting the entire method if a subclass needs to override it.

The chosen design was used as it breaks apart a long method (which in of itself is a code smell) into multiple methods with their own clear responsibility (e.g. detecting an enemy). This adheres to the SRP. As it is more modular, it improves readability and maintainability as it is easier to find a part of the logic to be changed. Subclasses that need to override some (e.g. isReady) but not all of the methods may also do so without needing to rewrite logic for the unchanged methods, promoting extensibility and adhering to DRY as well.

While there is a connascence of execution (call `getTargetDestination` before `fireProjectileAt`), the Turret abstract class ensures that it will execute in this order, and its subclasses can simply call super tick method to use this. They do not need to handle calling these methods and ensuring the correct order again, which avoids potential errors.

Decision 3: Who should be moving Projectiles?

Chosen design: Projectiles will move themselves when their tick method is called every turn.

Advantages	Disadvantages
Each Projectile is clearly responsible for their own movement.	Based on how the GameMap ticks each location, after moving some tiles, a projectile the GameMap may still call tick on it again, in the same turn, causing it to move again. This unintended behaviour must be addressed.

Alternative design 1: The Turret that fired the Projectile will handle its movement every turn too.

Advantages	Disadvantages
Easy way to ensure the GameMap doesn't cause projectiles to move again in the same turn, since the GameMap will only call Turret's tick method once.	If the Turret ground gets replaced, GameMap will no longer call its tick method. In turn, its projectiles fired will no longer be moved and is just stuck there permanently. This is unintended behaviour.
-	Turret also has an additional responsibility of handling the movement of its projectiles.

Alternative design 2: Projectiles don't have to move. They just hit their target instantly, so they don't even appear on the map. (This will not be discussed further, as explained in its disadvantage)

Advantages	Disadvantages
Would not have any issues from the above designs.	Removes the velocity component of projectiles, which as a feature would be less interesting. This is why it was proposed to have variable velocity rather than just instant hit.

The chosen design was used as Projectiles would be clearly responsible for moving themselves, acting independently from the entity that fired it. This adheres to the SRP. If the game was extended to have, for example, player-held guns, they will just need to spawn a projectile with a set destination, without needing to duplicate code for handling projectile movement, adhering to DRY as well. Furthermore, this avoids the Feature Envy code smell the alternative design has. In the alternative design, Turret would have a moveProjectile method that excessively interacts with Projectile's methods, like isStopped and getCloserDestination. This is a sign that the logic of moving projectiles should belong to Projectile itself.

Note: Since there is unintended behaviour for this design, it needs to be addressed. Because of how the GameMap calls tick on each location, if a

Projectile was moved anywhere downwards, OR in the same y-coordinate, anywhere rightwards, the GameMap will call the Projectile's tick method (causing it to move) again. As a workaround, if the Projectile detects that it moved in such a way, it will "lock" itself, so that when tick is called, all it does is "unlock" itself without executing the rest of the logic. Since the engine itself must not be changed, this is to address the limitation of the game engine while keeping the proposed feature more interesting.